

SNES bootup sequence (the llvm-mos-65816 platform)

How a `snes / snes-far` ROM goes from power-on to `main()`. This is a **reference** for the existing behavior; the *why* of the native-mode design lives in the plans linked at the end. Source of truth: [platforms/snes/crt0.c](#), [platforms/snes/link.ld](#), [platforms/snes/header.s](#), and the common chain merged via [platforms/snes/CMakeLists.txt](#).

TL;DR

The 65816 powers on in **6502-emulation mode**, reads the **emulation RESET vector** at `$FFFC` → `_start` (= the start of `.text` = `.init.50`). The crt0 preamble switches to **65816 native mode** and pins a known machine contract (E/M/X/SP/DBR/DP), then the **common init chain** (`.init.100` → `.200` → `.300`) sets up the soft stack, copies `.data`, clears `.bss`, and runs constructors, then `.call_main` calls `main()`. On the SNES there is no OS to return to, so a return from `main()` lands in an infinite **exit loop**.

The whole sequence at a glance

Power-on / reset
65816 in 6502-emulation
mode (E=1)

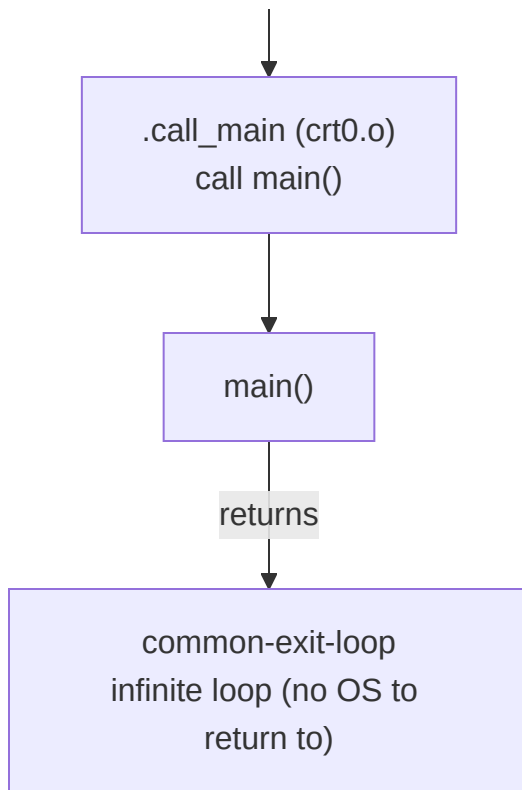
CPU reads emulation
RESET vector @ \$FFFC
→ `_start`

`.init.50 (crt0.c)` — \$8000
native-mode preamble:
SEI/CLD → XCE → 16-bit
TXS → SEP #\$30 →
PHK/PLB (DBR=0) → PPU
force-blank

`.init.100 (init-stack.S)`
`__do_init_stack` — point
the soft stack at \$2000

`.init.200 (copy-data / zero-
bss)`
copy `.data` ROM → RAM,
clear `.bss` (+ zp variants)

`.init.300 (init-array.c)`
run `__init_array`
constructors (if any)



`.init.*` fragments are emitted into `.text` and ordered by their numeric suffix, so the **section number is the boot order**. Unused fragments are garbage-collected (e.g. `hello.c` pulls in only `.init.50`, `.100`, `.200` -zero-zp-bss, `.call_main` — no `.data` to copy, no constructors).

Stage 0 — power-on and the reset vector

The 5A22 (the SNES's 65816) comes out of reset in **emulation mode (E=1)**, so it fetches the **emulation** RESET vector at `$FFFC`. The linker points it at `_start`, which is the very first byte of `.text`:

```
$8000  _start = .                ← .text base = reset entry
$8000  crt0.o:(.init.50)         0x18    ← native-mode preamble (this is _start)
$FFFC  SHORT(_start)            ← emulation RESET vector
```

(LoROM maps ROM `$8000-$FFFF` to file offset `$0000-$7FFF`, so `$FFFC` is file offset `$7FFC`.)

Stage 1 — `.init.50`: native-mode preamble (`crt0.c`)

The 24-byte fragment that establishes the machine contract. Source:

```
asm(".section .init.50,\"axR\",@progbits\n"
    " sei\n"                          // mask IRQ
```

```
" cld\n" // binary mode (decimal flag undefined at reset)
" clc\n" // clear carry, then exchange it with E:
" xce\n" // XCE -> E=0, 65816 native mode (M=1,X=1 kept)
" rep #$10\n" // REP #$10 -> 16-bit index regs (so txs takes 16 bits)
" ldx #$01ff\n" // LDX #$01ff (16-bit immediate; an 8-bit txs => SP=$00FF)
" txs\n" // hardware stack pointer -> $01FF (page 1)
" sep #$30\n" // SEP #$30 -> M=1,X=1: 8-bit A+index (codegen default)
" phk\n" // PHK -> push program bank (=0; reset code is bank $00)
" plb\n" // PLB -> DBR := 0 (explicit; abs globals + MMI0 read DBR:addr)
" lda #$00\n" // A = $00 for the NMITIMEN store below
" sta $4200\n" // NMITIMEN: no NMI/IRQ/auto-joypad
" lda #$8f\n" // A = $8F (bit 7 = force-blank, brightness 0)
" sta $2100\n"); // INIDISP: force blank, brightness 0
```

crt0 is built with `-mcpu=mosw65816 -fno-lto` (set in `platforms/snes/CMakeLists.txt`), so the 65816-only ops are plain mnemonics. `-fno-lto` is required because module-level inline `asm()` under LTO does not receive the `w65816` subtarget feature; with it, `ldx #$01ff` assembles to the correct 16-bit immediate (`a2 ff 01`), byte-identical to the old hand-encoded `.byte` form.

What actually executes (the linked bytes)

```
78 d8 18 fb c2 10 a2 ff 01 9a e2 30 4b ab a9 00 8d 00 42 a9 8f 8d 00 21
SEI CLD CLC XCE REP#10 LDX#$01ff TXS SEP#30 PHK PLB LDA#0 STA $4200 LDA#8F STA $2100
      L X→16 L 16-bit | L M=1 | | L NMITIMEN L INIDISP
      SP value | X=1 | L DBR := 0 off force-blank
      SP=$01FF PB pushed (=0)
```

`dev/run.sh crt0native` asserts this byte sequence (and the runtime DBR=0) as a standing gate.

The native-mode contract this leaves

Register	Value after <code>.init.50</code>	How / why
E	0 (native)	XCE — required for 16-bit accumulator/index codegen (<code>+mos-a16 / +mos-xy16</code>)
SP	\$01FF	page-1 hardware stack via a transient 16-bit <code>ldx #\$01ff; txs</code> (an 8-bit <code>txs</code> would set <code>\$00FF</code> and collide with the direct page)

Register	Value after <code>.init.50</code>	How / why
M	1 (8-bit A)	<code>SEP #30</code> — the codegen default; 16-bit regions are bracketed by <code>rep/sep #20</code>
X	1 (8-bit index)	<code>SEP #30</code> ; 16-bit-index regions bracketed by <code>rep/sep #10 (+mos-xy16)</code>
DBR	0	<code>PHK</code> ; <code>PLB</code> — explicit, so the 8-bit <code>abs / R_MOS_ADDR16</code> global path and the MMIO writes (which read <code>DBR:addr</code>) land in bank 0. Reset already leaves <code>DBR=0</code> ; making it explicit means a later bank switch / <code>MVN / MVP</code> / interrupt can't silently break the invariant. The native-16 <code>long / R_MOS_ADDR24</code> path is DBR-independent.
DP	0	reset default; the direct page never moves on this platform (imaginary registers + zero page live at <code>\$0000</code>)
PPU	force-blank, NMI/IRQ off	<code>INIDISP=\$8F</code> , <code>NMITIMEN=\$00</code> — a known display state with interrupts masked through bring-up

Stage 2 — the common init chain

Pulled in by `CMakeLists.txt` (`merge_libraries(snes-crt0 common-copy-data common-init-stack common-zero-bss common-exit-loop)`), each piece contributing a numbered `.init` fragment that runs after `.init.50` :

Order	Fragment	Source	Does
<code>.init.100</code>	<code>__do_init_stack</code>	<code>common/crt0/init-stack.S</code>	point the soft stack at <code>__stack = \$2000</code> (it grows down through low WRAM)
<code>.init.200</code>	copy <code>.data</code> , clear <code>.bss</code>	<code>copy-data.c</code> , <code>copy-zp-data.c</code> , <code>zero-bss.c</code> , <code>zero-zp-bss.c</code>	copy initialized <code>.data</code> from its ROM image (LMA) into RAM (VMA); zero <code>.bss</code> and the zero-page <code>.bss</code>
<code>.init.300</code>	<code>__init_array</code>	<code>init-array.c</code>	run C++ static constructors / <code>__attribute__((constructor))</code> (omitted if none)

Order	Fragment	Source	Does
—	.call_main	crt0.o	jsr main

The **soft stack** (the C call/local stack) is deliberately separate from the 65816 **hardware stack** (\$0100-\$01FF). Locals/spills use the soft stack via the `__rc0/__rc1` pointer; `JSR / RTS` return addresses use the hardware stack. This is why a page-1 hardware stack (`SP=$01FF`) is sufficient even in native mode.

Stage 3 — `main()` and exit

`.call_main` calls `main()`. On a console there is no OS to return to, so `common-exit-loop` makes a return from `main()` fall into an **infinite loop** (the program "ends" by spinning). Bring-up test ROMs typically compute a `corpus_result` / `sentinel` global and then `for (;;) {}` so an emulator can settle and read the value out of WRAM.

Memory map at boot ([link.ld](#))

\$0000-\$001F	imaginary (zero-page) registers <code>__rc0..__rc31</code>	(the GPR/Imag8/Imag16 file)
\$0020-\$00FF	zero page / direct page	
\$0100-\$01FF	hardware stack (JSR/RTS, interrupts)	SP set to \$01FF
\$0200-\$1FFF	low WRAM: soft stack grows DOWN from \$2000;	.data / .bss / heap grow UP from \$0200
\$2000-\$7FFF	(bank \$00) PPU/CPU MMIO, etc.	\$2100 INIDISP, \$4200 NMITIMEN
\$8000-\$FFFF	cartridge ROM (LoROM): .text/.init, .rodata, .data LOAD image	header @ \$FFB0, vectors @ \$FFE0-\$FFF

`snes-far` adds a second ROM bank (`$018000-$01FFFF`) for cross-bank far **data**, read via DBR-independent `long / [dp]` addressing; code stays in bank `$00`, so the boot sequence is identical (the `crt0` is inherited via `PARENT snes`).

Interrupt & reset vectors ([link.ld](#))

The 65816 has **two** vector tables — one used in native mode, one in emulation mode. The machine boots in emulation mode (→ `$FFFC` RESET); after `XCE` any interrupt would use the native table, but interrupts stay masked through bring-up (`SEI + NMITIMEN=0`), and both tables point NMI/IRQ at the same stubs.

```
Native ($FFE0-$FFEF): $FFE6 BRK→irq $FFEA NMI→nmi $FFEE IRQ→irq (COP/ABORT→0)
Emulation ($FFF0-$FFFF): $FFFA NMI→nmi $FFFC RESET→_start $FFFE IRQ→irq
```

The default handlers are **weak** bare-`rti` stubs (`crt0.c`), overridable by defining `nmi` / `irq` in user code. A bare `rti` is width-safe (it restores `P`, hence M/X, on return).

For a future real handler (out of scope today): a native-mode ISR can be entered with **M=0 and/or X=0** (it may interrupt a `rep #30` region), so it must save `P` and force known widths before touching A/X/Y, and restore via `rti`. The current stubs sidestep this by doing nothing while interrupts are masked.

Cartridge header (`header.s`)

Not executed, but part of the boot image: the LoROM internal header at `$FFB0-$FFDF` (title "LLVM-MOS SNES", map mode `$20` = LoROM/slow, ROM size `$05` = 32 KiB, NTSC). The checksum/complement at `$FFDC/$FFDE` are placeholders patched post-link by `tools/snes-checksum.py`.

References

- Native-mode entry rationale + the 8-bit-safe argument: `2026-06-14-321-native-mode-crt0.md`.
- Explicit DBR=0 contract (the `phk; plb`) + the addressing/relocation analysis: `2026-06-18-321-native-mode-crt0-xy16.md`.
- Standing contract gate: `dev/run.sh crt0native` (`dev/crt0native.sh`).
- Build/test mechanics & the addressing/DBR note: `agent-handoff.md`.